Tips och ledtrådar till Laboration 5

Laboration 5 innehåller flera delar som behöver fungera tillsammans. Om du fastnar är ett bra arbetssätt att dela upp problemet och arbeta med **en funktion i taget**.

Försök gärna lösa respektive del själv innan du använder ledtrådarna på denna sida.

Ett möjligt arbetssätt är att börja med att få studentkortet att fungera och därefter lägga till validering, historik och lagring.

OBS. Fastnar du länge på en del. Var inte rädd för att fråga en lärare om hjälp!

---

1. Börja med eventlyssnarna

Applikationen behöver reagera på olika saker som användaren gör.

Fundera på vilka event som är lämpliga när:

- formuläret skickas
- användaren klickar på knappen **Rensa**
- användaren klickar på knappen **Radera historik**

En eventlyssnare skapas med `addEventListener()`:

```
element.addEventListener("event", function () {
    // Kod som ska köras
});
```

Formuläret

När formuläret skickas vill vi hantera informationen med JavaScript istället för att låta webbläsaren skicka formuläret och ladda om sidan.

För detta kan du använda formulärets `submit`-event och event-objektets `preventDefault()`:

```
form.addEventListener("submit", function (event) {
    event.preventDefault();

    // Vad behöver göras här?
});
```

Fundera sedan på **vilka funktioner som behöver anropas och i vilken ordning**.

---

2. Läs information från formuläret

DOM-elementen för formulärfälten finns redan hämtade i startfilen.

För att läsa vad användaren skrivit i ett formulärfält används egenskapen `value`:

```
const value = inputElement.value;
```

För textfält kan det även vara användbart att använda `trim()` för att ta bort eventuella mellanslag i början och slutet:

```
const value = inputElement.value.trim();
```

Försök först att läsa värdena och skriva ut dem med `console.log()` om du är osäker på om du får rätt information.

---

3. Validera formuläret

Innan ett studentkort skapas behöver du kontrollera att de obligatoriska fälten innehåller värden.

I startfilen finns arrayen:

```
let errors = [];
```

Börja valideringen med att tömma tidigare fel. Kontrollera därefter varje obligatoriskt fält.

Ett exempel på en kontroll kan se ut så här:

```
if (someInput.value.trim() === "") {
    errors.push("Ett felmeddelande");
}
```

Du behöver göra motsvarande kontroll för de obligatoriska fälten i uppgiften.

När samtliga fält har kontrollerats kan du undersöka hur många fel som finns:

```
errors.length
```

Funktionen `validateForm()` ska returnera ett boolean-värde:

```
return true;
```

eller:

```
return false;
```

Fundera på **när respektive värde ska returneras**.

4. Visa felmeddelanden

Om valideringen hittar fel ska samtliga felmeddelanden i `errors` visas i listan på sidan.

Ett bra första steg är att rensa tidigare felmeddelanden:

```
errorList.innerHTML = "";
```

Därefter behöver du gå igenom arrayen `errors`.

Du kan exempelvis använda en vanlig `for`-loop eller `for...of`.

För varje fel behöver ett nytt `<li>`-element skapas:

```
const liElement = document.createElement("li");
```

Text kan läggas till med:

```
liElement.textContent = "...";
```

och elementet kan därefter läggas till i fellistan med:

```
errorList.appendChild(liElement);
```

Försök kombinera dessa delar så att **ett nytt `<li>` skapas för varje fel i arrayen**.

---

5. Skapa studentkortet

När valideringen lyckas ska informationen från formuläret visas på studentkortet.

DOM-elementen för studentkortet finns redan tillgängliga i startfilen.

Du kan ändra texten i ett element med:

```
element.textContent = "Ny text";
```

Kom ihåg att informationen ska hämtas från formuläret och inte skrivas in direkt i JavaScript-koden.

Ändra typsnitt

Det valda typsnittet finns i formulärets `<select>`-element.

CSS-egenskaper kan ändras från JavaScript genom `style`:

```
element.style.fontFamily = font;
```

Fundera på vilka delar av studentkortet som ska använda det valda typsnittet.

---

6. Skapa ett objekt för historiken

När ett giltigt studentkort har skapats ska informationen även läggas till i historiken.

Ett studentkort kan representeras som ett objekt:

```
const studentCard = {
    fullname: "...",
    email: "...",
    phone: "...",
    font: "..."
};
```

Värdena ska förstås komma från formuläret.

Historiken finns i arrayen:

```
let history = [];
```

Det senaste studentkortet ska visas **först** i historiken.

Fundera därför på vilken array-metod som passar bäst:

```
push()
```

eller:

```
unshift()
```

Vad är skillnaden mellan dessa?

---

7. Spara historiken i localStorage

`history` är en array med objekt. Den kan inte lagras direkt i `localStorage` på ett användbart sätt eftersom Web Storage lagrar värden som text.

Först behöver arrayen omvandlas till JSON:

```
JSON.stringify(history)
```

Ett värde kan sedan sparas med:

```
localStorage.setItem("nyckel", värde);
```

Du behöver alltså kombinera `JSON.stringify()` och `localStorage.setItem()`.

Ett bra sätt att tänka är:

```
JavaScript-array
      ↓
JSON.stringify()
      ↓
text
      ↓
localStorage
```

---

8. Läs historiken från localStorage

När sidan öppnas behöver tidigare sparad historik hämtas.

Data hämtas från `localStorage` med:

```
localStorage.getItem("nyckel");
```

Om det finns sparad information får du tillbaka en textsträng. Den behöver omvandlas tillbaka till JavaScript-data med:

```
JSON.parse(...)
```

Flödet blir alltså det omvända:

```
localStorage
      ↓
text
      ↓
JSON.parse()
      ↓
JavaScript-array
```

Tänk på att det kanske **inte finns någon tidigare historik** första gången användaren öppnar sidan.

Kontrollera därför att du faktiskt fått tillbaka något innan du använder `JSON.parse()`.

---

9. Visa historiken i DOM

När arrayen `history` innehåller studentkort behöver dessa visas på sidan.

Börja med att rensa tidigare visad historik så att samma poster inte läggs till flera gånger:

```
historySection.innerHTML = "";
```

Gå därefter igenom `history`.

Du kan exempelvis använda:

```
history.forEach(function (studentCard) {
    // Skapa och lägg till element
});
```

eller:

```
for (const studentCard of history) {
    // Skapa och lägg till element
}
```

För varje studentkort behöver du skapa de HTML-element som ska visas.

Exempel:

```
const article = document.createElement("article");
const name = document.createElement("p");

name.textContent = studentCard.fullname;

article.appendChild(name);
historySection.appendChild(article);
```

Bygg vidare på samma princip för övrig information.

Du behöver **inte skapa all HTML på en gång**. Börja gärna med att bara visa namnet. När det fungerar kan du lägga till e-postadress, telefonnummer och typsnitt.

---

10. Uppdatera historiken direkt

När ett nytt studentkort skapas räcker det inte att endast spara informationen i `localStorage`.

Historiken på sidan ska också förändras direkt.

Fundera därför på programmets flöde:

```
Skapa ett nytt studentkort
        ↓
Lägg till objektet i history
        ↓
Spara history i localStorage
        ↓
Visa history på nytt
```

Du har redan funktioner för flera av dessa steg.

Försök därför att **återanvända funktionerna** istället för att skriva samma kod på flera platser.

---

11. Rensa formuläret

Ett HTML-formulär kan återställas med:

```
form.reset();
```

Utöver formuläret behöver eventuella felmeddelanden tas bort.

Fundera på vilken funktion som redan ansvarar för felmeddelandena och vilka variabler eller DOM-element som behöver återställas.

Kom ihåg att knappen **Rensa inte ska radera historiken**.

---

12. Radera historiken

Ett värde kan tas bort från `localStorage` med:

```
localStorage.removeItem("nyckel");
```

Men kom ihåg att informationen finns på **flera ställen** i programmet:

```
localStorage
history-arrayen
DOM
```

När historiken raderas behöver dessa hållas synkroniserade.

Fundera på vilka befintliga funktioner du kan återanvända för att uppdatera sidan efter att arrayen har tömts.

Felsök en del i taget

Om programmet inte fungerar, försök först ta reda på **vilken del som inte fungerar**.

Använd gärna `console.log()` för att kontrollera exempelvis:

```
console.log(fullnameInput.value);
console.log(errors);
console.log(history);
```

Du kan även kontrollera innehållet i `localStorage` via webbläsarens utvecklarverktyg.

Några bra frågor att ställa när du felsöker är:

- Körs min eventlyssnare?
- Får jag rätt värde från formulärfältet?

- Innehåller `errors` de fel jag förväntar mig?
- Returnerar valideringsfunktionen rätt värde?
- Läggs objektet faktiskt till i `history`?
- Vad innehåller `history` före och efter `JSON.stringify()`?
- Finns informationen i `localStorage`?
- Har jag uppdaterat DOM efter att datan förändrats?

Försök hitta **var i flödet resultatet slutar bli det du förväntar dig**.

Ta en del i taget

Du behöver inte få hela laborationen att fungera direkt.

En möjlig arbetsordning är:

1. Fånga formulärets `submit`-event.
2. Läs och skriv ut formulärvärden med `console.log()`.
3. Få studentkortet att uppdateras.
4. Lägg till valideringen.
5. Visa felmeddelanden i DOM.
6. Skapa ett objekt för ett studentkort.
7. Lägg objektet i `history`.
8. Visa `history` på sidan.
9. Spara `history` i `localStorage`.
10. Läs tillbaka historiken när sidan öppnas.
11. Implementera Rensa.
12. Implementera Radera historik.

Testa efter varje steg. Om något slutar fungera vet du då ungefär vilken förändring som orsakade problemet.